1. Technical scope and execution environment

CMPE 249 Track A - technical methods and software documentation. This report consolidates the completed experimental protocol, evaluator, runtime safeguards, analysis methods and script responsibilities. It contains no Python source listings or script appendices. Code availability: the author will add a GitHub repository URL and commit or release identifier before publication.

Three supplied COCO-pretrained checkpoints were evaluated without training or fine-tuning: YOLOv8n, YOLO11n and RT-DETR-L. The first two represent the YOLO detector family; RT-DETR supplies a transformer-based set-prediction family [2,3,5,6,9]. The comparison is between checkpoint configurations of unequal capacity, not a controlled architecture-only experiment.

The persistent verified run is results_verified/20261103_track_a within the project. Dataset preparation, synthetic evaluator checks, GPU smoke checks, six primary evaluations, timing sweeps, labeled ablations, analysis and packaging were executed locally. A later 19-image supplement added untimed visual prediction passes while reusing the original timing measurements.

The existing dtect virtual environment was used for inference and metrics. No CUDA/PyTorch upgrade was made for this run. Workers set PyTorch CPU threads to four and use GPU device 0. The system is a Windows laptop using WDDM. Its recorded power scheme was Silent, and the initial nvidia-smi snapshot showed a 55 W power-cap field and other desktop GPU processes. The results therefore describe this measured laptop state, not an exclusive-GPU or maximum-performance deployment.

Versions, device properties, checkpoint hashes, image manifests, command history and evaluator outputs are saved with the run. PDF documents were generated separately using ReportLab in the bundled document runtime; the inference environment was preserved.

Component	Recorded value
GPU	NVIDIA GeForce RTX 5070 Ti Laptop GPU
Device memory	11.91 GiB reported by PyTorch
OS	Windows-11-10.0.26200-SP0
Python	3.13.6
PyTorch / CUDA build	2.14.1+cu130 / 13.0
Ultralytics / torchvision	8.4.198 / 0.29.1+cu130
pycocotools / NumPy	2.0.11 / 2.5.4
OpenCV / Pillow	5.0.0.93 / 13.0.0
Plotting / report PDF	Matplotlib 3.11.2 / ReportLab 4.4.9

2. Detector architecture and image preprocessing

Version-specific implementation inspection used the installed Ultralytics 8.4.198 source and model YAMLs. Copies and SHA-256 hashes of the relevant files are stored under implementation_snapshot. Official research papers and framework documentation provide context [2,3,5,6,8,9]; the local source snapshot establishes the precise inference path used.

YOLOv8n uses Conv/C2f feature-extraction blocks, SPPF and feature fusion by upsampling, concatenation and downsampling before a three-scale Detect head. YOLO11n substitutes C3k2 blocks and adds C2PSA, retaining P3/P4/P5 detection. Their DetectionPredictor applies class-aware NMS, then restores boxes to original image coordinates. The inspected YOLO loss path uses TaskAlignedAssigner, BCE classification and BboxLoss with distributional regression; these describe training design only because no training was performed.

The supplied RT-DETR-L YAML uses HGStem/HGBlock, AIFI within-scale interaction, RepC3 cross-scale fusion and RTDETRDecoder. It should not be described as an assumed ResNet checkpoint. The inspected loss path uses Hungarian set matching and class, box/GIoU, auxiliary and denoising terms. RTDETRPredictor filters confidence and caps decoder outputs without NMS; its normalized center/size coordinates are converted to corners and scaled by original width and height.

Primary input is a batch-one 3-channel square 512 x 512 tensor, with rect=False. For YOLO, the aspect-preserving resize factor is the smaller of 512/W and 512/H, followed by padding to the square. RT-DETR uses scale_fill=True and stretches the image independently along each axis. The nominal resolution is shared, but image distortion and padded content differ. Framework preprocessing reorders BGR to RGB, changes BHWC to BCHW, transfers to the GPU and normalizes pixel intensities by 255.

FP32 is verified through model-parameter and input-tensor dtypes, rather than inferred from the half=False flag. Forward hooks also assert CUDA placement, batch one, evaluation mode and disabled gradients; integrity checks confirm observed square dimensions. FP16 sweep cases repeat dtype verification. These checks do not establish a separately controlled TF32 hardware policy or cross-device bitwise determinism.

Checkpoint	Parameters before fusion	Predictor / suppression
yolov8n.pt	3157200	DetectionPredictor; class-aware NMS
yolo11n.pt	2624080	DetectionPredictor; class-aware NMS
rtdetr-l.pt	32970476	RTDETRPredictor; NMS-free

3. Datasets, taxonomy and coordinate validation

COCO uses the full val2017 split: 5,000 images, 80 categories and 36,781 instance annotations [1]. Images and official annotations were reused from the existing local dataset. Original image IDs, sparse category IDs, area metadata and crowd flags are retained. Model class names are resolved to annotation category IDs; contiguous detector class indices are not treated as official COCO IDs.

KITTI uses 500 images from the existing prepared Ultralytics validation split of 1,496 images. The parent file list is sorted; Python random.Random(42).sample selects 500 entries, and the chosen entries are sorted for execution. All models reuse that exact manifest. A separate seed-42 sample of 200 entries from these 500 supplies labeled ablations. Hashes, source paths, parent-split details and counts are recorded. This prepared split is not an official KITTI test-server submission [4].

The primary shared vocabulary is person/pedestrian, car and truck. Converted KITTI IDs are assigned within its own export and need not equal the native COCO IDs. Per-class names, rather than coincident numeric IDs, establish the semantic pairing. Bicycle/cyclist remains an explicitly approximate supplementary mapping because a cyclist box can include both rider and bicycle.

Prepared KITTI labels contain class, normalized center x/y and normalized width/height. Conversion uses x1=(xc-w/2)W, y1=(yc-h/2)H, x2=(xc+w/2)W and y2=(yc+h/2)H. After validating finite values and positive extents, corners are clipped to image bounds and exported as pixel [x1,y1,x2-x1,y2-y1]. Dimensions come from the actual decoded image. Missing files raise errors; an existing empty file is retained as a legitimate negative. The selected 500-image subset had zero missing and zero empty label files.

KITTI van, person_sitting, tram and misc annotations are excluded from the mapped scored vocabulary. Nonmapped predictions are excluded from shared-class scoring. A mapped prediction over an excluded object can still be a false positive. DontCare, truncation, occlusion and difficulty fields are absent from the prepared YOLO labels, so no official ignore-region or difficulty filtering is claimed. These are COCO-style scores on converted KITTI annotations.

COCO concept / native ID	KITTI name / source ID	KITTI export ID	Selected KITTI GT
person / 1	pedestrian / 3	1	296
car / 3	car / 0	2	1860
truck / 8	truck / 2	3	79
bicycle / 2	cyclist / 5	4	114

Count / definition	Value
COCO shared GT: person / car / truck	11,004 / 1,932 / 415
KITTI primary shared GT	2235
Ablation primary GT: person / car / truck	120 / 740 / 40 (900 total)
KITTI supplementary cyclists	114 in 500 images; 38 in ablation
Object area	COCO metadata unchanged; KITTI clipped box width x height

4. Official metrics and saved evaluation evidence

Both datasets use pycocotools.COCOeval with iouType=bbox and useCats enabled [7]. COCO all-class results use 80 categories. The separate shared evaluation uses person, car and truck on both datasets. Supplementary bicycle/cyclist scores are exported separately. The raw KITTI all/ evaluator directory includes four mapped concepts; the primary KITTI table and shared/ results use only the three primary classes.

Intersection-over-union is intersection area divided by union area. A detection must satisfy the evaluator matching rule at the requested IoU threshold and class. COCOeval handles score ordering, one-to-one assignment and crowd behavior; no custom AP matcher replaces it. Predictions are saved before evaluation, permitting repeat metric calculation without a GPU inference pass.

AP50-95 averages precision over ten IoU thresholds from 0.50 through 0.95 at intervals of 0.05 and 101 recall thresholds from 0 to 1. AP50 and AP75 retain one IoU threshold. Per-class AP averages valid entries of the precision array for that class and all-area/maxDets=100. The shared macro uses the same three concepts in both domains. An absent class or undefined size bin is represented as null/N/A, not zero.

Export retains up to 300 predictions per image. The official evaluator uses maxDets=[1,10,100]; AP uses the 100 setting, while AR1, AR10 and AR100 describe the corresponding maximum-detection settings. AR averages recall across evaluator thresholds and eligible categories. Export capacity and evaluator limits are different and are both recorded.

Small/medium/large use COCOeval area ranges with boundaries at 32 squared and 96 squared pixels. COCO retains its original metadata, commonly based on segmentation area; converted KITTI uses clipped box area. Therefore size AP/AR is reproducible within each dataset but does not establish equivalent physical object-size groups across domains.

A prediction record stores image_id, category_id, original-pixel xywh bbox and score. Ground truth retains image dimensions, category definitions, annotation IDs, area and iscrowd. Saved outputs include summary.txt, metrics.json, per_class.csv and evaluator_arrays.npz. The arrays retain precision, recall, scores, IoU/recall thresholds and category/image IDs. These support regenerated tables and plots; they are not claimed to contain every in-memory COCOeval object.

Metric family	Reported outputs / interpretation
Precision	AP50-95, AP50, AP75; all-area, per-class and size-stratified
Recall	AR1, AR10, AR100 and size-stratified AR
Domain transfer	Paired per-class AP and three-class macro AP
Undefined values	null in JSON; N/A in report tables
Qualitative diagnostic matching	COCOeval IoU .50, confidence .25; TP/FP/FN only on labeled KITTI
19 local images	Timing and output behavior only; no labeled accuracy metrics

5. Experiment matrix and verification gates

The main planned matrix contains 38 configurations: six primary dataset/model evaluations, 20 timing cases and 12 labeled ablations. Timing has seven unique settings for each YOLO and six for RT-DETR. Each one-factor sweep starts from size 512, FP32, confidence .25 and YOLO IoU .70: resolution 384/512/640, precision FP32/FP16, confidence .10/.25/.50, and YOLO-only NMS IoU .50/.70. Identical baselines are deduplicated; this is not a full factorial design.

Primary accuracy uses size 512, FP32, batch 1, device 0, confidence .001, export cap 300 and YOLO IoU .70. The labeled 200-image ablation evaluates 384/512/640 FP32 and 512 FP16 at the accuracy confidence floor. For each model, the 512 FP32 ablation filters its compatible primary KITTI predictions to the 200 selected IDs. The other nine ablation cases require fresh inference.

Before primary inference, 15 assertion-based evaluator and annotation checks passed. Three GPU smoke jobs then used the first three selected KITTI images to verify each checkpoint, tensor dtype/device/batch, no-gradient evaluation state and output restoration. An early path-list batching issue was detected through observed tensors; a text-file image source uses the framework file loader with batch one.

Evaluator / data test	Expected behavior; all passed
perfect	AP approximately 1
empty_predictions	AP 0; valid empty result index
duplicate_detection	AP remains 1 after full recall; duplicate not a new TP
wrong_class	AP 0 for the labeled class
wrong_localization	AP 0 for disjoint localization
overlapping_gt_alternative_match	Two TPs at IoU .50 using an unmatched alternative GT
absent_gt_class	Undefined class AP is null
coordinate_conversion_and_clipping	Expected pixel box and clipping match
reversed_box	Reject reversed / zero-area extents
nonfinite_box	Reject nonfinite normalized coordinates
missing_label	Reject missing annotation file
empty_label	Accept a legitimate empty label file
unknown_category	Reject unknown label ID
unknown_image_id	Reject prediction image ID outside GT
duplicate_image_id	Reject duplicate GT image IDs

Completed checks	Evidence
Main experiment configurations	38 / 38 successful; no experiment OOM/failure
Integrity checks	41 / 41: manifests, counts, shape, timing means and AP macro
Cache compatibility	38 GPU jobs compatible; includes three smoke jobs
Saved-prediction reproduction	Exact shared metrics reproduced on a KITTI export

6. Timing, GPU memory and runtime isolation

The separate application-timing baseline uses confidence .25. The same 19 supported images are decoded into CPU BGR arrays before measurement. Each fresh worker loads one model and performs an initial setup prediction, then ten explicit warmup calls and five timed repetitions per image. This produces 95 timed calls per configuration, 285 across the three timing baselines and 1,900 across all 20 timing cases.

Full-prediction latency starts immediately before model.predict on a decoded image and ends after a CUDA synchronization following the returned Results object. CUDA is also synchronized before the start. The monotonic high-resolution CPU clock measures input adaptation, resize/padding or stretching, host-to-device transfer, channel conversion/normalization, model forward, postprocessing, coordinate restoration and framework/callback overhead. Disk reading, image decoding, CPU box export, plotting and image writing are outside this region.

Forward-only time comes from the framework inference profile, whose installed implementation synchronizes the accelerator around its timing boundary. It is a forward-region measurement, not an optimized CUDA-kernel-only or TensorRT benchmark. Full latency additionally includes preprocessing, postprocessing and API overhead. The verification hooks remain part of the measured prediction path.

Mean, median, p95 and sample standard deviation are retained for per-call full latency. Five repetition means and their sample standard deviation show run-to-run spread on the fixed image set. FPS equals 1000 divided by mean latency in milliseconds, equivalently timed calls divided by summed timed seconds. It is serial batch-one throughput under these boundaries, not camera-to-display FPS or maximum batched throughput.

After warmup, the CUDA allocator peak counters are reset. Peak allocated memory is the highest live tensor allocation; peak reserved memory includes allocator-held blocks. Both include model/runtime allocations in the worker and are reported in MiB. They do not represent incremental model-only memory or total VRAM use by the operating system and other applications. Each model/configuration runs in a fresh subprocess, which exits and releases its owned allocations.

The Silent power scheme, WDDM display use and visible repetition variability limit portability. Clocks, thermals and scheduling were not continuously controlled; no unrelated applications were killed and no power settings were changed. Small differences relative to repetition spread should not be presented as stable wins. No OOM occurred; the implemented recovery policy retries an OOM once in a clean process at identical settings and never silently switches device, precision or resolution.

Model	Mean / median / p95 ms	Forward ms	FPS	Allocated / reserved MiB
yolov8n.pt	10.59 / 9.49 / 11.16	7.78	94.45	40.4 / 62.0
yolo11n.pt	14.10 / 12.44 / 23.14	10.98	70.90	68.2 / 90.0
rtdetr-l.pt	27.27 / 26.16 / 34.41	24.61	36.67	237.2 / 302.0

7. Quantitative findings and interpretation

All numbers below come from saved evaluator and timing outputs; AP is expressed on [0,1]. Domain differences subtract the same three-class AP50-95 metric, never 80-class COCO AP from KITTI AP50. The primary shared KITTI scores use 2,235 GT objects across 500 images. Per-class scores and counts remain available to interpret a macro average.

RT-DETR-L gains 15.65 COCO AP percentage points over YOLOv8n at 2.58 times the measured full-prediction latency. YOLO11n gains 1.20 points at 1.33 times latency. These results compare model configurations with approximately 32.97, 3.16 and 2.62 million parameters respectively; they do not isolate the causal contribution of NMS, attention or any individual module.

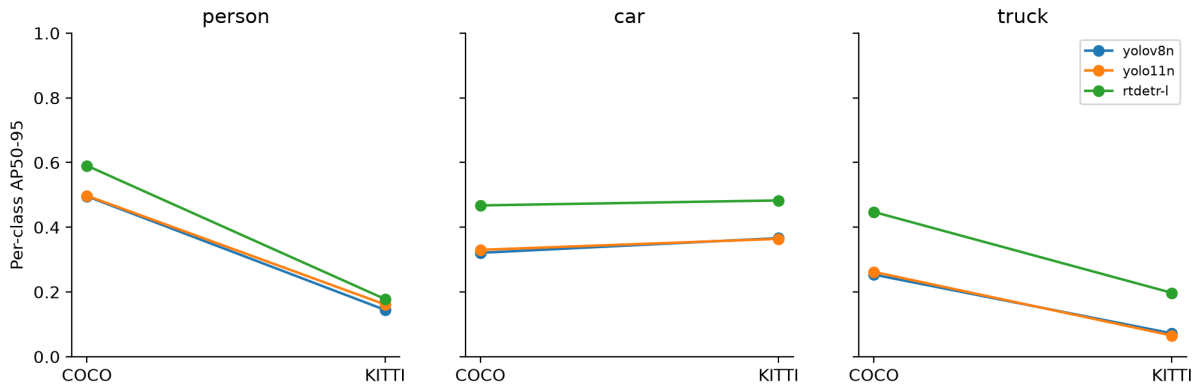
Shared macro AP drops on KITTI, but car AP increases by 4.54 points for YOLOv8n, 3.40 for YOLO11n and 1.55 for RT-DETR-L. Person and truck decreases therefore matter to the macro interpretation. Dataset composition, object scale, viewpoint and label conventions remain confounded. The supplementary KITTI bicycle/cyclist APs are 0.0037, 0.0017 and 0.0011 respectively; their very low values are disclosed alongside the semantic box mismatch.

The labeled-ablation table uses exactly the same 200 KITTI images and 900 primary GT objects in every row. Raising FP32 input size from 512 to 640 adds 2.32, 4.12 and 0.66 AP percentage points for YOLOv8n, YOLO11n and RT-DETR-L. The corresponding FP16 changes at size 512 are approximately -0.06, -0.07 and -0.19 points. These are fixed-subset effects without AP confidence intervals.

Some YOLO timing curves are not monotonic with resolution, and FP16 does not uniformly improve application latency. The data support reporting the measured configurations and variability, not assuming an expected hardware speedup. Output count changes in confidence/NMS sweeps measure filtering behavior; they do not establish improved precision, recall or accuracy.

Model	COCO 80-class AP / AP50	Shared COCO AP	Shared KITTI AP	Change (points)
yolov8n.pt	0.3518 / 0.4963	0.3567	0.1943	-16.24
yolo11n.pt	0.3638 / 0.5113	0.3632	0.1969	-16.63
rtdetr-l.pt	0.5082 / 0.6927	0.5017	0.2860	-21.57

Model	384 FP32 AP	512 FP32 AP	640 FP32 AP	512 FP16 AP
yolov8n.pt	0.1593	0.1871	0.2103	0.1865
yolo11n.pt	0.1637	0.1915	0.2327	0.1907
rtdetr-l.pt	0.2636	0.2952	0.3018	0.2933



8. Local 19-image study and qualitative analysis

The local benchmark_images folder contains exactly 19 supported image files; the twentieth directory entry is a Python utility, not an image. The utility was not used to infer scene provenance or labels. Inventory checks verify source dimensions and SHA-256 hashes. The images include animal pairs, cyclists, streets, wet-road reflections, traffic, overhead viewpoints and dense crowds, as established by visual inspection of the actual files.

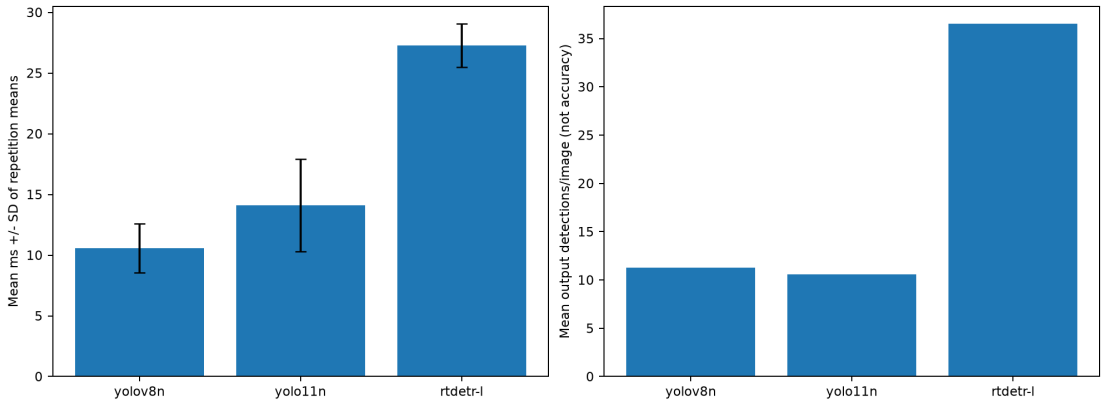
The dedicated supplement runs three fresh untimed GPU prediction cases, one per model, at the .25 timing/visual baseline. It produces 57 image/model rows, 169 nonzero image/model/class-count records, complete prediction JSON and 19 panels. Each panel contains the original and all three model outputs with class names and scores. Per-image timing summaries reuse the five samples from the original benchmark; image saving and drawing do not contaminate those measurements.

The 22-page supplement contains summary, timing heat map and inventory pages followed by one comparison page per image. Its tables link each image ID to the filename, dimensions, latency distribution, output counts and predicted classes. A per-image p95 is not emphasized with only five samples; the baseline p95 aggregates 95 calls per model. Full parameter-sweep summaries retain the 20 previously completed cases.

For traffic.jpg, YOLOv8n emits 22 boxes, YOLO11n 21 and RT-DETR-L 140 at confidence .25. In the dense-crossing image 19, totals are 9, 7 and 49. The inspected panels show denser low-confidence labels and differing localization or class assignments in RT-DETR outputs. These observations do not establish a false-positive rate, recall or correct-object count. No ground truth was supplied for this set, and confidence calibration differs between models.

Labeled qualitative analysis uses KITTI instead. Six scenes are selected by recorded rules: small-object count, crowding, pedestrian count, truck count, maximum GT overlap fraction, and highest shared YOLOv8n false-positive count. Official COCOeval assignments at IoU .50 and confidence .25 provide TP/FP/FN diagnostics there. FN counts treat GT matched only by below-threshold detections as unmatched at the display threshold. Crops and panels preserve image IDs, legends and common settings. Overlap alone is not proof of physical occlusion.

Artifact	Contents
benchmark_19_report.pdf	22 pages covering all 19 local images
tables/per_image_results.csv	57 image/model rows; timings, output counts and classes
tables/timing_samples.csv	285 original baseline timing samples
tables/class_counts.csv	169 nonzero class-count rows
experiments/qualitative_*	Three new GPU cases; predictions and tensor evidence
figures/image_01.jpg ... image_19.jpg	Original and three-detector comparison panels



9. Script responsibilities and code availability

The repository should identify run_track_a.py as the authoritative experiment entry point. The following inventory documents responsibilities only; source code belongs in the linked GitHub repository. Keep the verified_experiment directory structure intact because the modules import shared utilities. The configuration, model checkpoint filenames and image manifests are part of reproducibility.

Code repository: to be supplied by the author. Recommended citation fields are repository URL, exact commit or release tag, environment lock file and run ID 20261103_track_a. No repository has been published or linked automatically.

Script / supporting file	Responsibility
run_track_a.py	Primary entry point. Plans cases; orchestrates stages, isolated workers, caching, logging and OOM recovery.
verified_experiment/core.py	Dataset discovery/audit, KITTI label conversion, manifests, GT overlays, official COCOeval export and 15 smoke assertions.
verified_experiment/worker.py	Single-model CUDA worker. Enforces dtype/device/batch/eval checks; exports predictions or synchronized repeated timing samples.
verified_experiment/verify_run.py	Checks image hashes, expected counts, prediction limits, AP macro consistency, timing means and processed shapes.
verified_experiment/check_resume.py	Reconstructs input/config/checkpoint/worker signatures and checks compatible completed GPU jobs without inference.
verified_experiment/recompute_metrics.py	Reevaluates a saved prediction JSON and asserts equality with the saved shared-class metrics.
verified_experiment/reporting.py	Aggregates tables and plots, computes domain/parameter effects, creates KITTI panels, snapshots source and verifies ZIP bundles.
verified_experiment/report_content.py	Builds the main report narrative and tables from saved metrics, keeping PDF and LaTeX content aligned.
verified_experiment/render_report.py	Renders the main report with ReportLab and checks its PDF page count.
benchmark_19_report.py	Creates the dedicated unlabeled-image supplement using existing timings, separate qualitative workers and inspected scene notes.
verified_experiment/render_benchmark_report.py	Renders the 19-image summary, inventory and per-scene comparison pages.
build_technical_methods_report.py	Builds this documentation from completed outputs; runs no model inference and embeds no script source.
Configuration and scene metadata	verified_experiment/config.json stores paths/protocols. benchmark_scene_notes.json contains inspected descriptions, not labels or GT.
Historical files - not verified entry points	run_pipeline.py, evaluate_kitti_mapped.py and run_verified_pipeline.py are retained earlier versions; they did not supply the new verified metrics.

10. Reproduction, deliverables and limitations

Stage order is prepare, smoke, coco, kitti, benchmark, ablation, analyze and package. Stages can run separately. The run directory retains resolved configuration, environment/checkpoint hashes, dataset/image manifests, planned matrix, append-only events/commands, per-attempt worker logs, predictions, evaluator arrays, tables, figures and reports. Reproduction can start from saved predictions without repeating GPU inference. The tested packaged metric command reproduced every saved shared KITTI metric exactly.

Compatible inference caches use the case configuration, image metadata and current content hashes, category definitions, checkpoint hash, and core/worker code hashes. A changed signature invalidates the previous completion marker rather than silently treating it as current. Earlier metadata-only signatures were upgraded only after the original preparation hashes and old code/config signatures matched. The cache does not independently guarantee an unchanged operating system, driver or every external dependency; inspect the environment manifest and use a new run ID after material environment changes.

Primary failures stop the affected stage; optional sweep/ablation failures are recorded and allow other cases to continue. No experiment inference failure or OOM remained in the completed matrix. Early UTF-8 parsing, NumPy JSON serialization and loader-batching issues were corrected and recorded. Fifteen evaluator checks, 41 run-integrity checks and 38 resume-compatibility checks passed. Main and supplement ZIP archives passed member-integrity tests.

Reporting limits: prepared KITTI lacks official ignored-region/difficulty data; bicycle/cyclist is approximate; area conventions differ; capacity and preprocessing are unequal; timing is a single-laptop sample under the recorded Silent plan. FP32 dtype checks are not cross-hardware determinism guarantees. No labeled accuracy is claimed for the local 19 images, no threshold was selected as an unbiased optimum, and no confidence interval for AP was estimated.

AI-use statement. An AI coding agent implemented and executed the local workflow, inspected implementation and output artifacts, and generated reports from saved outputs. The student is not represented as having independently checked code, sources or results. The author should review the report, add the GitHub URL and pinned revision, and retain the saved evidence. This document contains technical descriptions and filenames only; no Python listings are included.

References: original research and official sources previously checked during the experiment; the installed source snapshot is the authority for version-specific runtime details.

[1] Lin et al. Microsoft COCO: Common Objects in Context. ECCV 2014. https://arxiv.org/abs/1405.0312

[2] Carion et al. End-to-End Object Detection with Transformers. ECCV 2020. https://arxiv.org/abs/2005.12872

[3] Zhao et al. DETRs Beat YOLOs on Real-time Object Detection. CVPR 2024. https://arxiv.org/abs/2304.08069

[4] Geiger, Lenz and Urtasun. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. CVPR 2012. https://www.cvlibs.net/publications/Geiger2012CVPR.pdf

[5] Ultralytics. YOLOv8 model documentation. https://docs.ultralytics.com/models/yolov8/

[6] Ultralytics. YOLO11 model documentation. https://docs.ultralytics.com/models/yolo11/

[7] COCO Consortium. Official Python COCOeval implementation. https://github.com/cocodataset/cocoapi/blob/master/PythonAPI/pycocotools/cocoeval.py

[8] Ultralytics. Official framework source repository. https://github.com/ultralytics/ultralytics

[9] Ultralytics. RT-DETR model documentation. https://docs.ultralytics.com/models/rtdetr/